

TP6 : Création et alimentation du Datamart Ventes

Création d'un Datamart pour Suivre les Ventes.

1. Spécification des dimensions et de la table de fait
2. Structuration du Datamart

Alimentation du Datamart

1. Extraction des données à partir de la base de données
2. Transformation
3. Chargement au niveau des tables Ventes, Ventes_temp, ventes_products, vente_customers
4. Gestion des erreurs & Ordonnancement des sous jobs.

Objectif

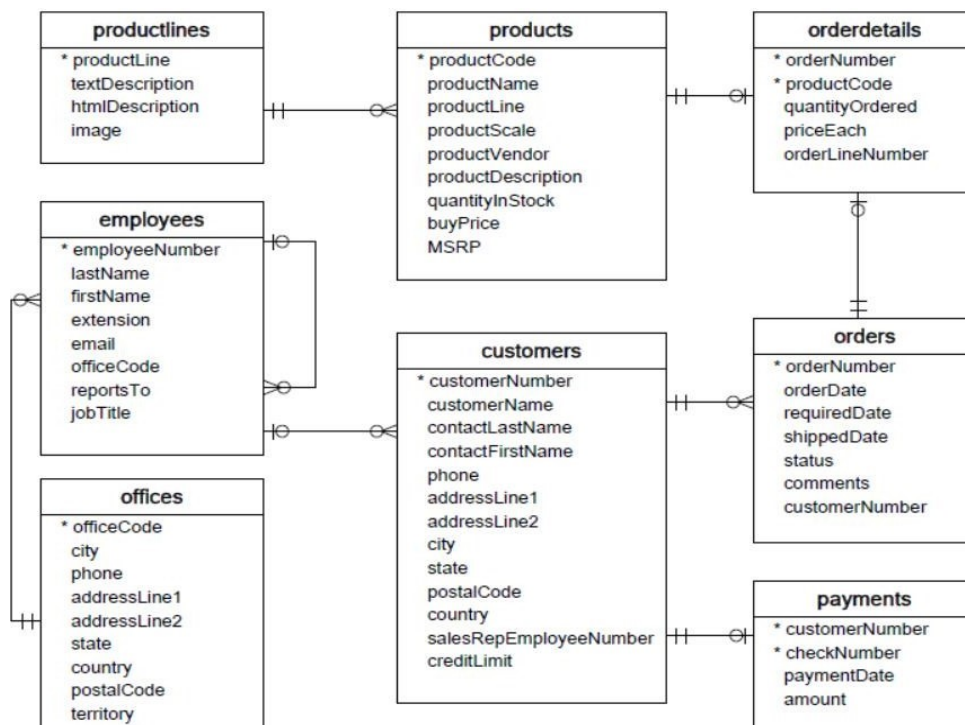
Une entreprise de fabrication de miniatures souhaite mettre en place un système d'information décisionnel sous la forme d'un datamart (un entrepôt de données) pour observer son activité de ventes sur plusieurs axes (par client, par produits et par unité temporel).

De ce fait, on crée un entrepôt de donnée à partir de la base de données relationnel, pour analyser plus efficacement les ventes de cette société.

Création d'un Datamart pour Suivre les Ventes

1. Spécification des dimensions et de la table de fait

Nous avons les tables suivantes dans notre base de données et on voudra modéliser le datamart ou l'entrepôt de données des ventes afin de pouvoir calculer le nombre total des ventes et leurs montants.

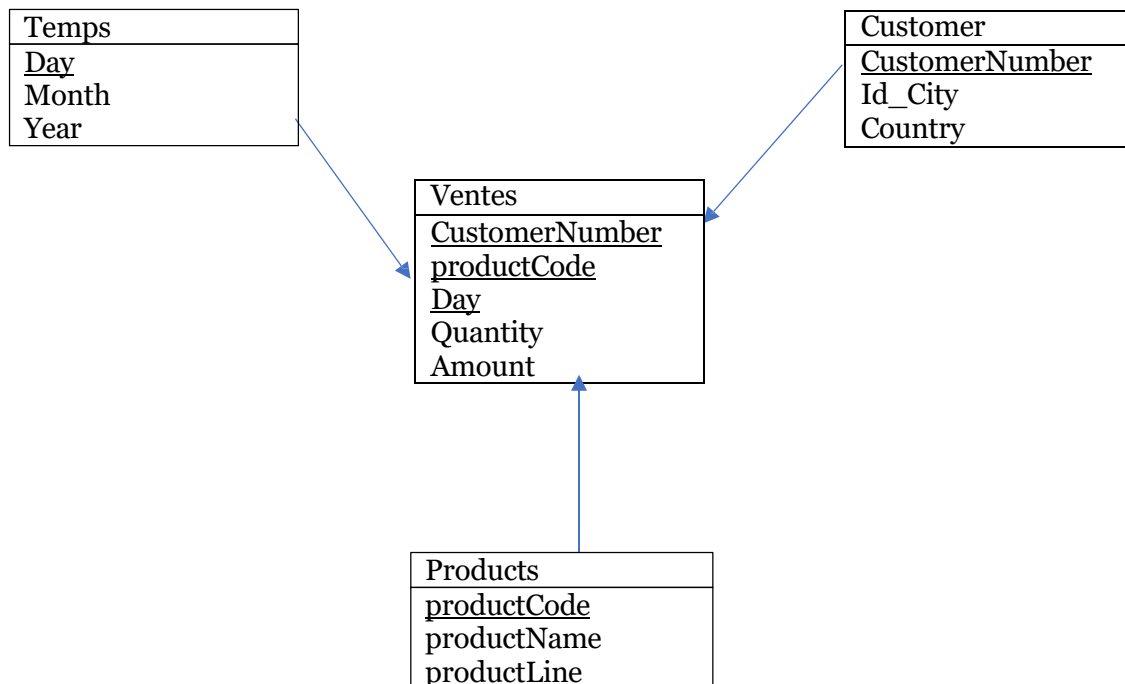


Dans notre Datamart, on voudra analyser les ventes sur plusieurs dimensions, notamment par unité de temps (jour, mois, année), par produit et aussi par client. De ce fait, on aura la table de fait « Ventes » et les dimensions suivantes :

- Dimension Temps
- Dimension Produits
- Dimension Clients

2. Structuration du Datamart

Après spécification de la table de fait et des dimensions, notre datamart aura la structure suivante (schéma en étoile) :



Rappel :

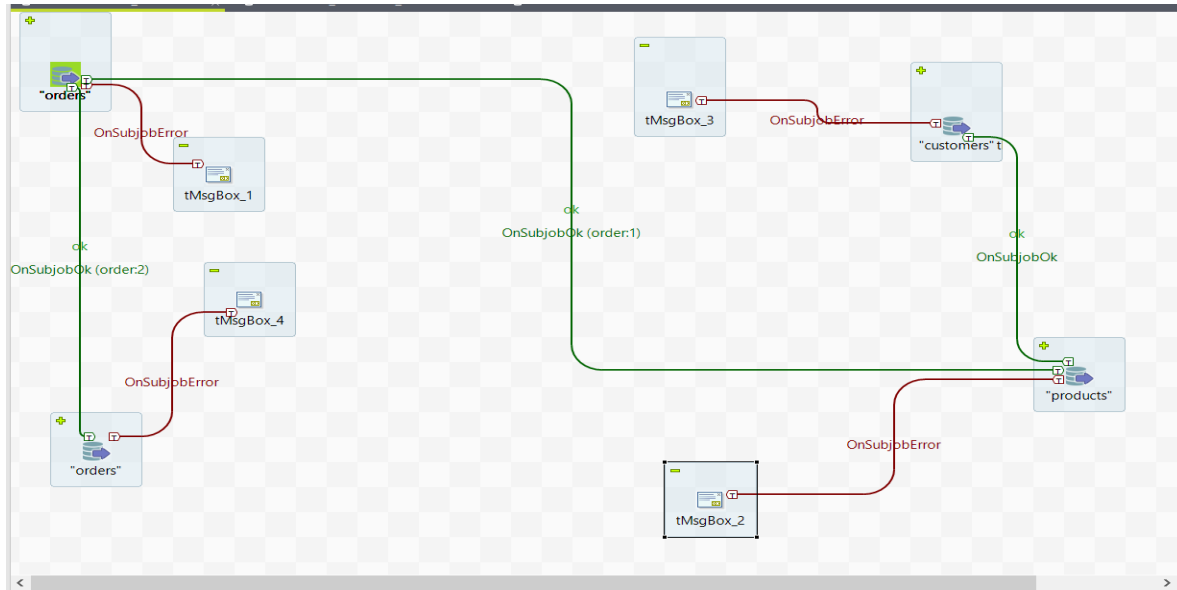
- Une **table de faits** contient les valeurs numériques qu'on désire mesurer (les faits : comme *quantité vendue* ou *montant des biens vendues*), elle contient les clés associées aux dimensions. Il s'agit des clés étrangères dans la table de faits
- Une **dimension** est une **table** qui contient les axes d'analyse (les **dimensions** comme le temps, client ou produits) selon lesquels on veut étudier des données observables (les faits : comme quantité vendue ou montant des biens vendues) qui donnent aux utilisateurs des renseignements nécessaires à la prise de décision

Alimentation du Datamart

Commencer par créer un nouveau job, nommer le Datamart_ventes.

1. Extraction des données à partir de la base de données

Les Jobs Talend d'Alimentation de la base de données Ventes



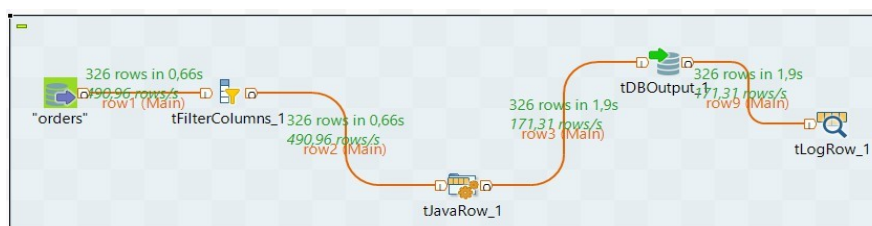
On va utiliser 4 sous-jobs talend dans le job crée dans l'ordre suivant pour alimenter les tables de la base de donnée ventes:

- Customers
- Products
- Temps
- Ventes

N.B : Créer une nouvelle base de données dans MySQL, nommer la 'Ventes', et créer une connexion à cette de base de données dans Talend pour y créer toutes les tables de l'entrepôt

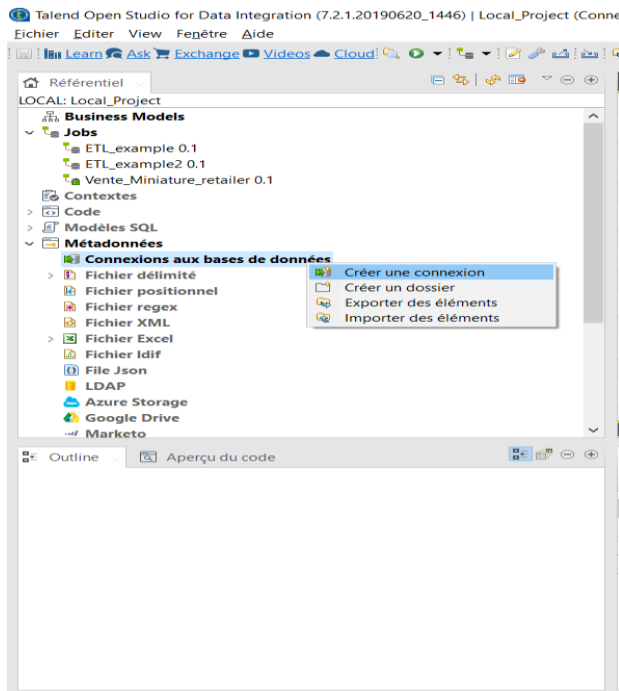
2. Transformation

2.1. Dimension Temps

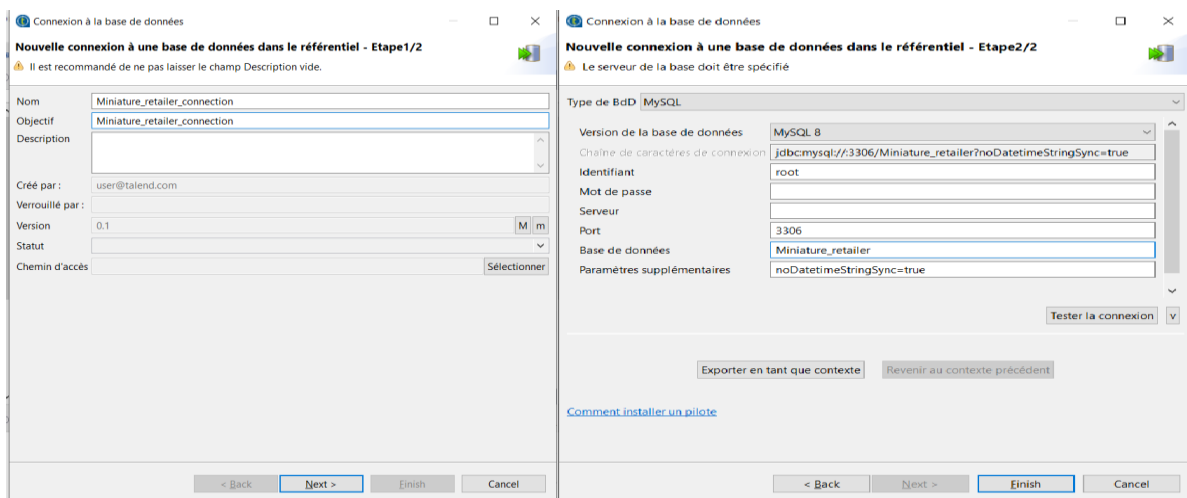


Commencer tout d'abord par l'ajout de la table source pour cela créer d'abord la connexion avec la base de données ensuite sélectionner et glisser la table 'orders' (Si la connexion est déjà créée glisser la table directement):

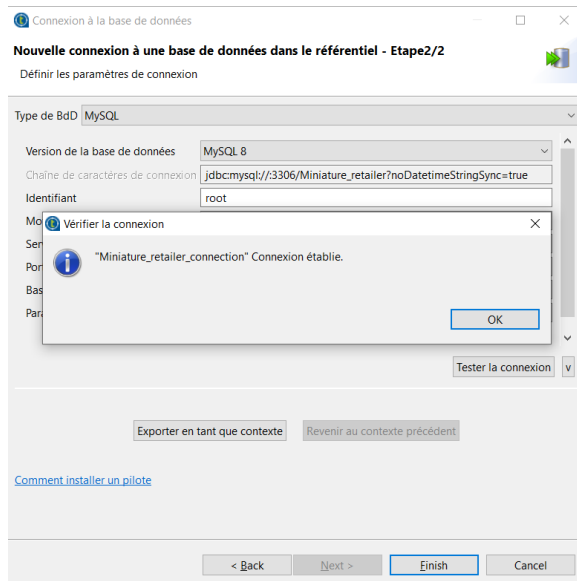
Dans le volet Référentiel, cliquer avec le bouton gauche sur Métadonnées ensuite cliquer sur 'Créer une connexion'



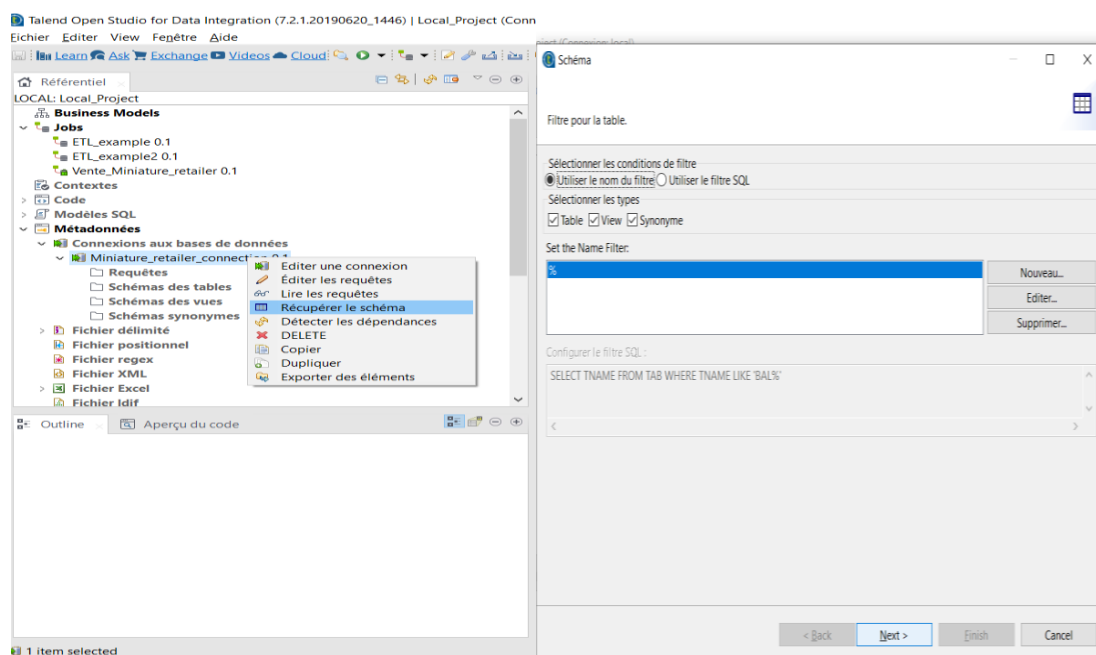
Nomme la connexion : Miniature_retailer_connection, cliquer sur suivant et renseigner les informations concernant la base de données Miniature_retailer



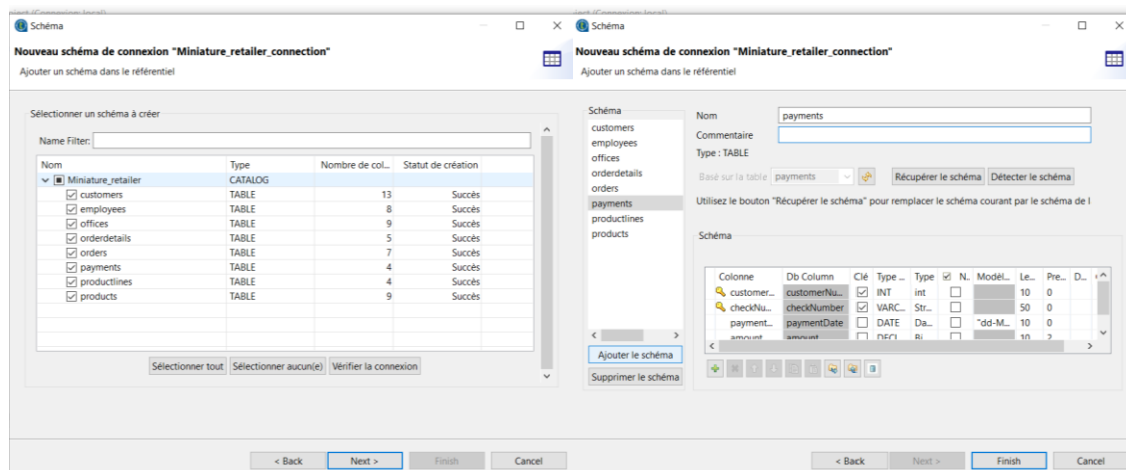
Tester la connexion en appuyant sur Tester la connexion.



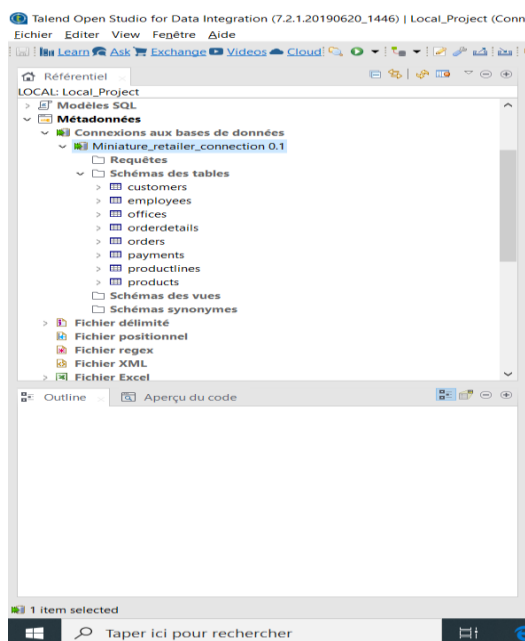
Pour peupler le schéma des tables, cliquer avec le bouton gauche sur la connexion créée ensuite sur Récupérer le schéma, ensuite cliquer sur suivant



Sélectionner la base de données Miniature_retailer et cliquer sur Next ensuite Finish.



Voilà, la connexion est peuplée par les tables de la base de données.



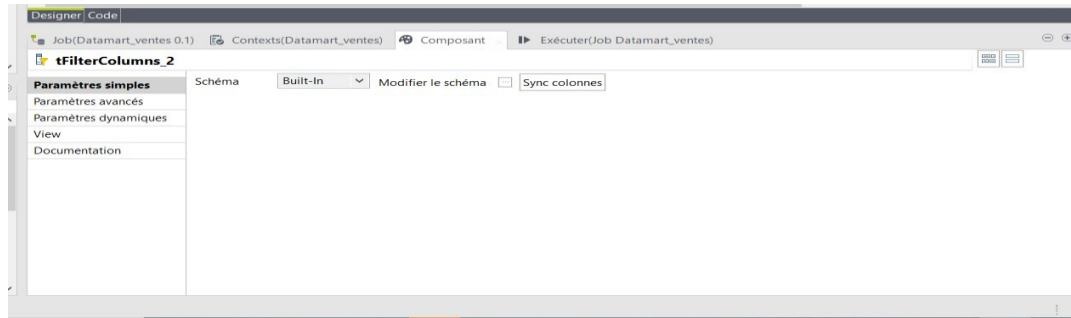
Ensuite sélectionner la table 'orders' et glisser la dans l'interface

Pour la dimension Temps, ajouter un tfilterColumns (à partir de la palette) pour ne sélectionner que les colonnes voulues, ensuite ajouter tJavaRow (à partir de la palette) pour générer les colonnes 'Month' et 'Year'.

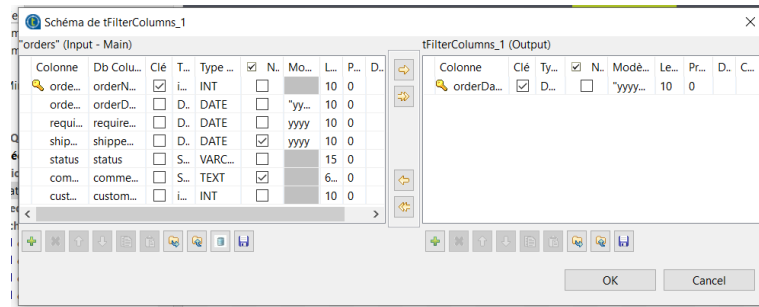
-Sélectionner tFilterColumns, appuyer sur modifier le schéma et configurer le comme suit :

- Cliquer sur tFilterColumns, ensuite schéma

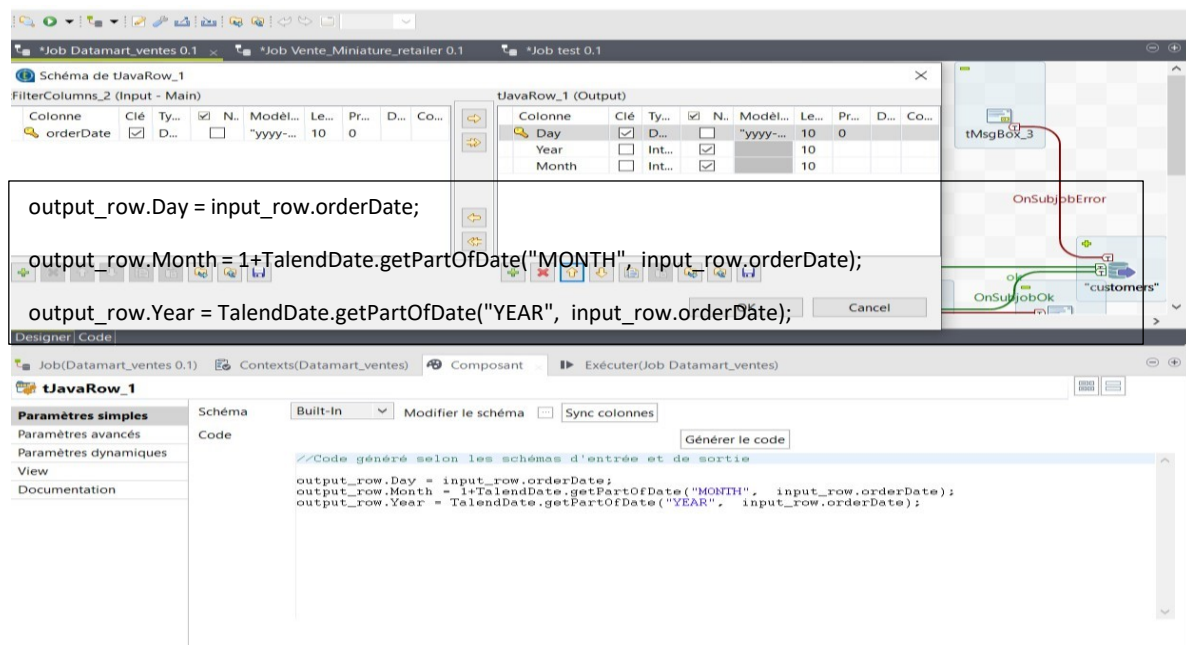
cliquer sur qui se trouve à côté de modifier le



- Seulement OrderDate qui sera transférer en output



-Ensuite sélectionner le tJavaRow, appuyer sur modifier le schéma (), faire passer la colonne OrderDate, renommer la en Day, ensuite créer deux autres colonnes Month et Year ; fermer la fenêtre et appuyer sur **GÉNÉRER le code** puis insérer le code suivant :

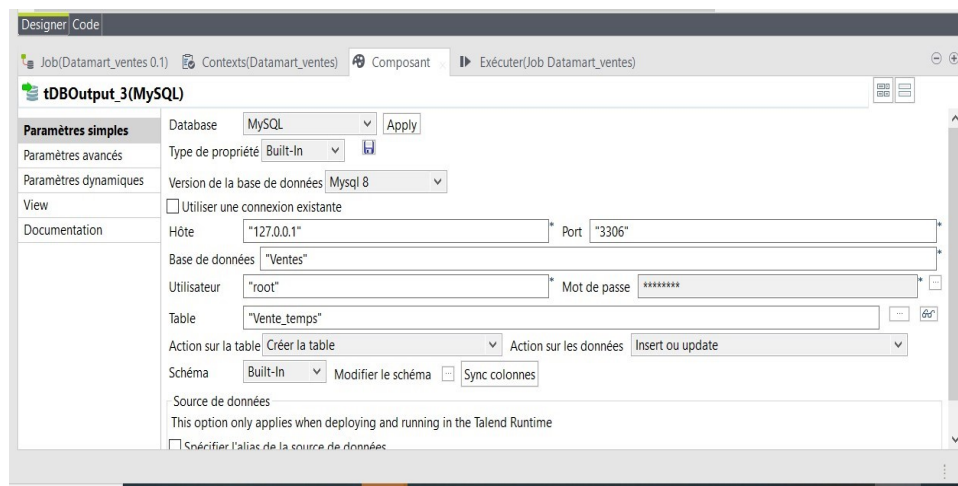



```
output_row.Day = input_row.orderDate;
output_row.Month = 1+TalendDate.getPartOfDate("MONTH", input_row.orderDate);
output_row.Year = TalendDate.getPartOfDate("YEAR", input_row.orderDate);
```

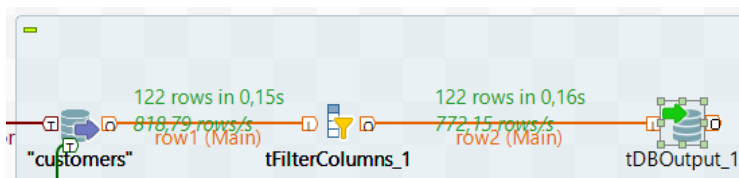
N.B : il faut toujours vérifier le format de la date de nos données sources 'yyyy-MM-dd' ou 'dd-MM-yyyy'

-Ensuite connecter tJavarrow a un tDBOutput ; Le tDBOutput doit être relié à la base de données **Ventes**. Pour cela créer une nouvelle base de données dans Mysql nommée **Ventes**.

Créer la connexion dans Métadonnées avec la base de données **Ventes** (comme celle réalisée avec la base de données Miniature_retailer)

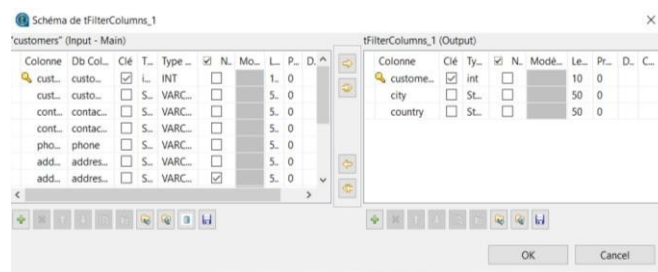


2.2 Dimension Customers



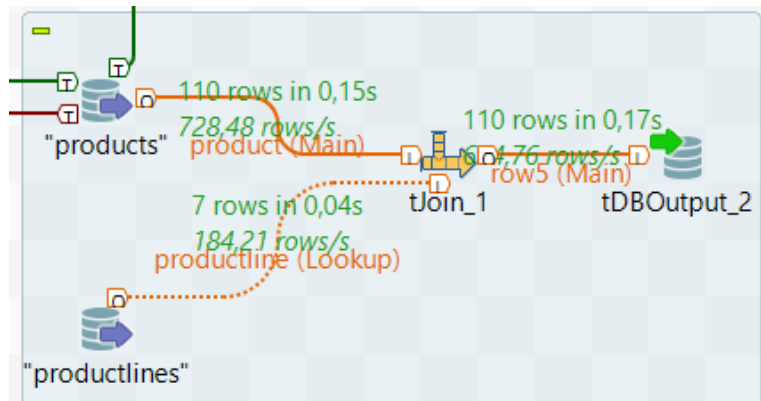
Pour la dimension Customers, ajouter un tfilterColumns pour ne sélectionner que les colonnes voulues.

- Sélectionner tFilterColumns, appuyer sur modifier le schéma et configurer le comme suit :



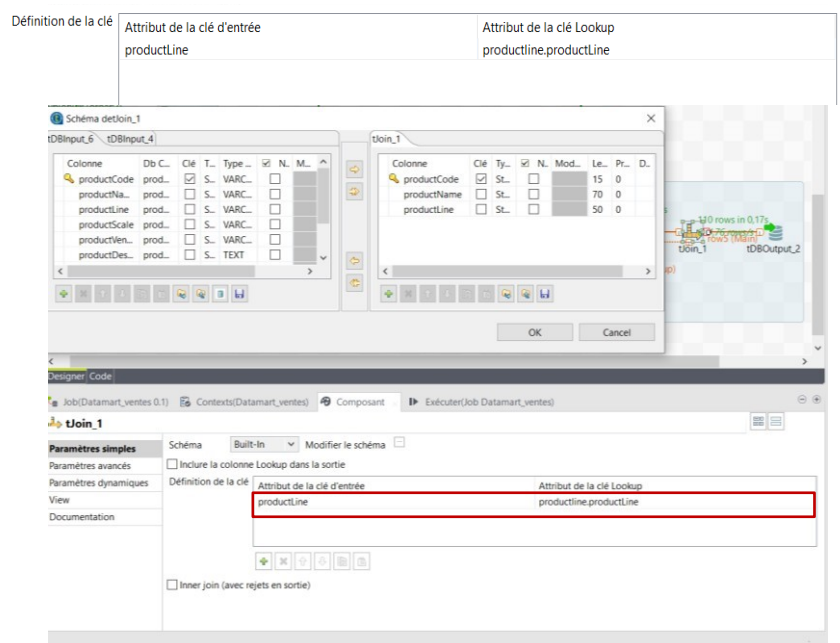
Ensuite ajouter un tDBOutput, le tDBOutput doit être relié à la base de données **Ventes** et la table est nommée **Ventes_customers**.

2.3 Dimension Products



Pour la dimension Products, ajouter un tJoin pour effectuer une jointure entre les tables Products et ProductLines.

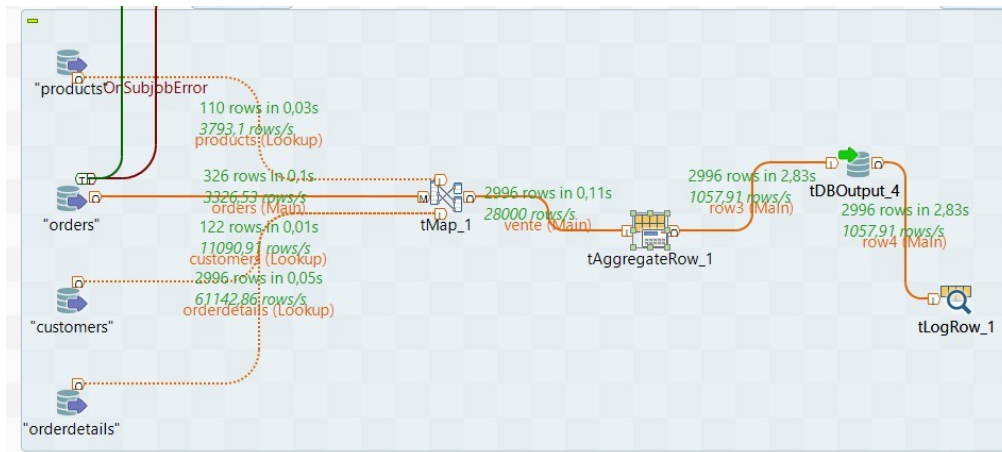
- Connecter la table products en *main* ensuite connecter la table productlines.
- Sélectionner tjoin, appuyer sur modifier le schéma, faire passer les champs : productCode, productName et productLine
- Faire correspondre dans la dimension clé, l'attribut de la clé d'entrée avec Attribut de la clé Lookup



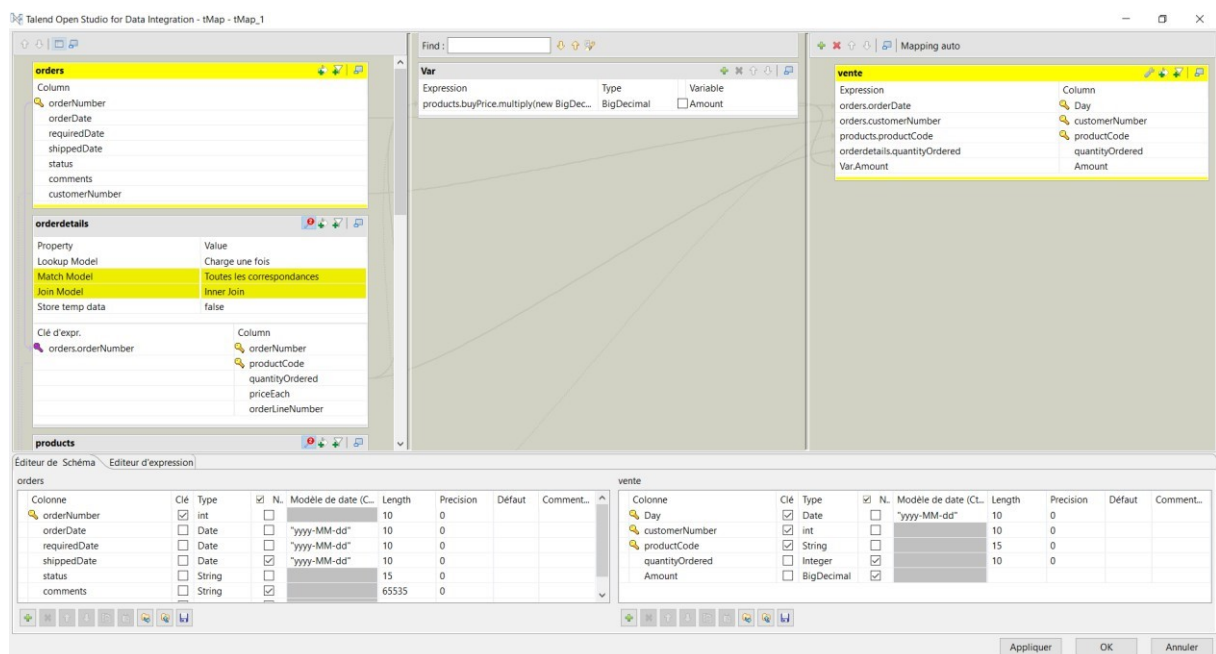
Ensuite ajouter un tDBOutput, le tDBOutput doit être relié à la base de données **Ventes** et la table est nommée *Ventes_products*.

2.4 Table de fait Ventes

Pour la table de fait Ventes, ajouter les tables products, orders, customers, et orderdetails.
Connecter orders en premier à tMap ensuite connecter les autres tables. Configurer tmap comme



suit :



Colonne	Cle	Type	N.	Modèle de date (C.L.)	Length	Precision	Défaut	Comment...
orderNumber	int				10	0		
orderDate	Date			"yyyy-MM-dd"	10	0		
requiredDate	Date			"yyyy-MM-dd"	10	0		
shippedDate	Date			"yyyy-MM-dd"	10	0		
status	String				15	0		
comments	String				65535	0		

Colonne	Cle	Type	N.	Modèle de date (C.L.)	Length	Precision	Défaut	Comment...
Day	int				10	0		
customerNumber	String				15	0		
productCode	Integer				10	0		
quantityOrdered	BigDecimal							
Amount								

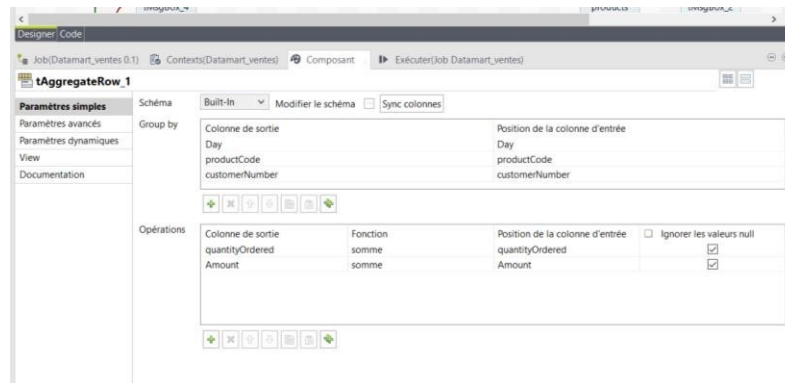
Créer une variable intermédiaire Amount de type BigDecimal dont l'expression est :

products.buyPrice.multiply(new BigDecimal(orderdetails.quantityOrdered))

A ne pas oublier avec le composant tMap :

- Spécifier la table orders comme la liaison source principale
- Joindre les différentes tables en glissant les clés primaires vers les clés étrangères.
- Spécifier le type de jointure comme inner join

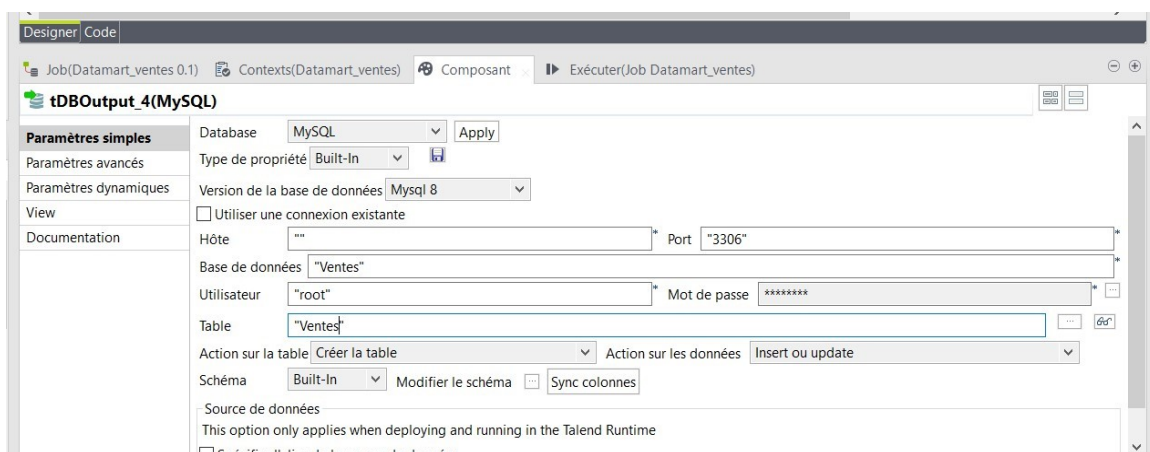
Ensuite ajouter un tAggregateRow, configurer le comme suit :



3. Chargement au niveau des tables Ventes, Ventes_temp, ventes_products, vente_customers

Dans chaque job, nous avons un tDBOutput pour charger les données dans la base de données Ventes. Pour chaque job nous spécifions dans tDBOutput :

- La base de données
- L'utilisateur
- Action sur la table
- Action sur les données

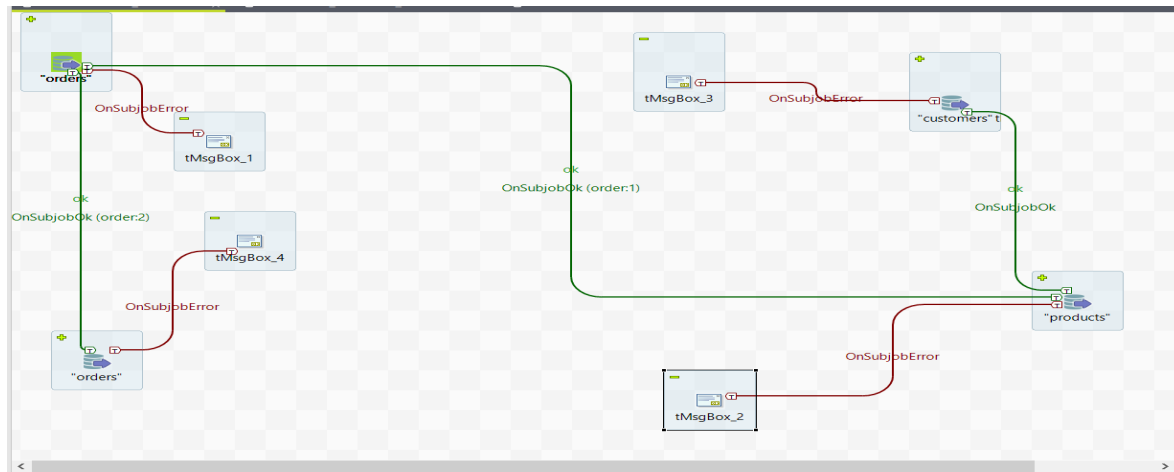


4. Gestion des erreurs & Ordonnancement des sous jobs.

Pour gérer les exceptions en cas d'erreurs lors de la connexion ou l'alimentation de la base de données :

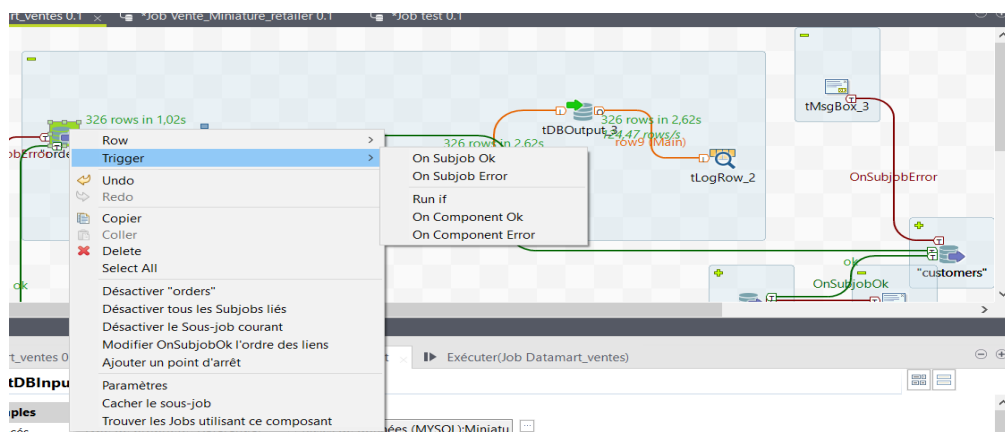
Mettre en place 2 types d'exceptions :

- OnSubjobOK si le sous job est terminé sans erreur et pour permettre l'ordonnancement,
- OnSubJobError en cas d'erreur du sous job



OnSubjobOK

Pour ordonnancer les sous jobs, cliquer sur l'entrée du job ; choisir trigger ensuite OnSubjobOK

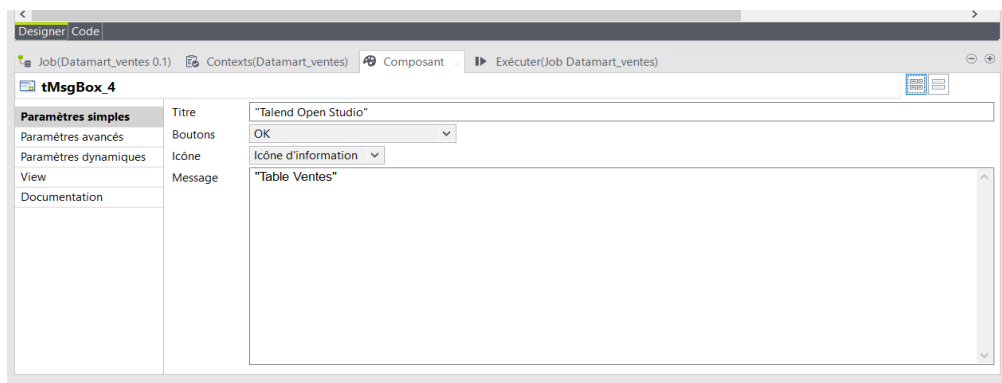


Connecter les sous jobs. Les Triggers mettent un ordre d'exécution entre les différents sous jobs.

OnSubjobError

De la même manière ajouter des onSubjobError pour chaque entrée de données, et connecter le à tMsgBox.

Dans le message de tMsgbox indiquer le nom du sous job comme suit :



N.B : Il faut paramétrer tMsgBox et tDBOutput pour tous les sous jobs.

Exécuter le job et vérifier les tables créées dans la base de données Ventes.